

COPILOTTRAINING
TECHNICAL BRIEFING

From Repo to Feature

Building Team Tools as You Go with Copilot in VS Code

Build the toolkit that makes every future feature faster.

GitHub Copilot · VS Code · 45 min

🤔 THE CORE QUESTION

"How does a team build a Copilot workflow that gets faster with every feature?"

Five capabilities chain into a repeatable workflow — and every pass leaves reusable assets.

Context, prompts, skills, plans, and risk review. All in .github/. All version-controlled.

Dev on a new codebase

Has Copilot but no project context — suggestions miss team conventions

Tech lead scaling a team

Domain knowledge stays tribal; standards erode as headcount grows

Dev iterating on a feature

Loses sprint cycles to incomplete tickets and unreviewed plans

Onboarding to first merged PR

Down from 5 days to 2 when AGENTS.md gives Copilot project context from day one

Mid-sprint context switches

Eliminated when evaluate-issue flags ticket gaps before a developer picks up the work

Architectural risks in review

/review-plan catches undiscovered gaps before a line of implementation code is written

TABLE OF CONTENTS

PART 1

Context First

AGENTS.md as the prerequisite

One /init prompt gives Copilot permanent project knowledge

PART 2

Build the Toolkit

Prompts + skills in .github/

Both artifact types are Markdown — both become slash commands

PART 3

Plan, Risk, and Ship

Plan first. Risk triage before code.

The severity table makes the go/no-go call explicit and visual

PART 4

The Compounding Effect

Toolkit grows with every feature

Four use cases with measurable outcomes — and a 15-min first step

PART 1

Context First

AGENTS.md is the prerequisite — every prompt that follows depends on it.

One Prompt

/init analyzes the workspace and drafts a thorough AGENTS.md

Instant Multiplier

30 seconds of setup improves every Copilot interaction after it

Lives in the Repo

Version-controlled context every developer on the team shares

```
$ git clone <repo> && code .
↳ One prompt. Permanent context.
```

Copilot Without Context Misses Project Conventions

❌ Without AGENTS.md

Generic suggestions that miss team naming conventions

Wrong file locations
Creates utils/ when the team uses services/

Convention violations caught in code review, not during generation

💡 The /init Prompt

Analyzes the workspace — stack, structure, scripts, conventions

Generates AGENTS.md and commits it to the repo root

30 seconds
for thorough, project-specific context

✅ With AGENTS.md

Convention-aware suggestions from the first prompt

Correct file locations, naming, and patterns inline

Code review focuses on logic, not style corrections

Permanent
context multiplier for every interaction

What /init Generates: Thorough Workspace Context

AGENTS.md

```
## Project Overview
React 18 + TypeScript nonogram puzzle SPA.
Target: puzzle enthusiasts and educators.

## Tech Stack
Vite 5 · CSS Modules · Vitest · npm
(No Tailwind. No Redux. Named exports only.)

## Commands
npm run dev    – localhost:5173
npm run test   – Vitest watch mode
npm run lint   – ESLint + TypeScript tsc

## Conventions
PascalCase components, co-located .module.css
Results pattern for errors (no thrown exceptions)

## Key Entry Points
New page → src/pages/ + App.tsx + Nav.tsx
New hook → src/hooks/ + index.ts export
```

⚡ Instant project awareness

Stack, commands, conventions, and entry points — all in one committed file

🔗 Shared by every developer

Committed to the repo root — version-controlled context no one has to recreate

📈 Permanent multiplier

Every subsequent Copilot prompt improves — not just the next one

The /init Prompt: What to Ask For

Copilot Chat → /init

```
/init Analyze this workspace and generate
thorough guidance for agents.

Include:
- Incorporate the lazy dev instructions from
  github.com/DietrichGebert/ponytail/AGENTS.md
- Project overview (purpose, domain, users)
- Tech stack: languages, frameworks, pkg mgr
- Folder structure + one-line descriptions
- Exact build/run/test/lint commands (real scripts)
- Coding conventions: naming, formatting, errors
- Architectural decisions + component interactions
- Setup steps, env vars, non-obvious gotchas
- Entry points for common tasks (route, test)

Concise, repo-specific, no boilerplate.
```

🔗 Bootstrap from a reference

Incorporating ponytail/AGENTS.md seeds lazy-developer conventions before the analysis runs

🎯 Real scripts, not examples

Explicitly asks for actual package.json scripts — prevents 'npm run build' boilerplate

🧹 Concise by design

The closing line keeps output tight — AGENTS.md stays a quick reference

PART 2

Build the Toolkit

Humans Run Prompts. AI run skills. Both are just Markdown files in .github/.



Prompts

/evaluate-issue checks any ticket for completeness in seconds



Skills

/create-spa-page builds a fully wired page per team conventions



Accumulate

Each file in .github/ becomes a slash command the team inherits

```
/create-prompt and /create-skill
↳ Save once. Invoke forever.
```

Humans Run Prompts. AI run skills.

Prompts (.prompt.md)

Structured evaluation
Check completeness, correctness, readiness

Parameterized with {{ISSUE}} or {{PLAN}}

Saved to .github/prompts/
Becomes /evaluate-issue, /review-plan, etc.

🛠 Skills (.skill.md)

Domain-encoded action
Encodes how the team creates pages — not generic boilerplate

Parameterized with {{PAGE_NAME}}, {{DESCRIPTION}}

Saved to .github/skills/
Becomes /create-spa-page, /add-puzzle-type, etc.

evaluate-issue.prompt.md — Completeness Checker for Any Ticket

.github/prompts/evaluate-issue.prompt.md

```
---
description: Evaluate a bug report or feature
  request for completeness before work begins.
---
For Bug Reports, check:
- [ ] Problem statement clearly described?
- [ ] Expected behavior stated?
- [ ] Reproduction steps: numbered + repeatable?
- [ ] Severity: how many users affected?

Return:
**Type** · **Completeness** · **Missing elements**
**Suggested additions** · **Ready to implement?**

Issue: {{ISSUE}}
```

Reusable across projects

Same prompt works for any issue in any codebase — not tied to the SPA

Invokable as /evaluate-issue

Saved to .github/prompts/ — available as a slash command immediately

Catches gaps before work starts

Missing repro steps, expected behavior, severity — surfaced before a dev picks it up

Create the Prompt. Then Use It. Two Commands.

Copilot Chat

```
# 1. Create the prompt (saves to .github/prompts/evaluate-issue.prompt.md)
/create-prompt this prompt evaluates a bug or issue or feature
  request for completeness and correctness.

# 2. Run it against a real ticket — immediately
/evaluate-issue “Puzzles don’t show me what I’ve done wrong,
  only a ‘keep trying’ message.”
```

📁 Step 1: File created

evaluate-issue.prompt.md written to .github/prompts/ and invokable as /evaluate-issue

🔍 Step 2: 1 of 5 criteria met

4 gaps: expected behavior, repro steps, environment, severity

↩ Four questions to reporter

Ticket returns for completion — spec is closed before a developer picks it up

create-spa-page.skill.md — Domain-Encoded Page Creation

.github/skills/create-spa-page.skill.md

```
---
description: Create a new SPA page — component,
  CSS module, route, nav link, and test.
---
Files to create:
1. src/pages/{{PAGE_NAME}}/{{PAGE_NAME}}.tsx
2. src/pages/{{PAGE_NAME}}/{{PAGE_NAME}}.module.css
3. src/pages/{{PAGE_NAME}}/{{PAGE_NAME}}.test.tsx

Files to update:
4. Add <Route> in src/App.tsx (RR v6 pattern)
5. Add <NavLink> in src/components/Nav/Nav.tsx

Follow src/pages/Home/Home.tsx as reference.
PAGE_NAME: {{PAGE_NAME}}
```

5 files, zero decisions

Component, CSS module, test, route, nav link — all created in one invocation

Encodes team conventions

PascalCase, CSS Modules, named exports, reference pattern — all baked in

Any developer, senior output

New team members invoke the skill and ship convention-correct code from day one

Create the Skill. Then Use It. Two Commands.

Copilot Chat

```
# 1. Create the skill (saves to .github/skills/create-spa-page.skill.md)
/create-skill to create new pages in the SPA app

# 2. Use it to generate a new page — immediately
/create-spa-page Add a “how to play nonograms” page
  that is linked to from the home page
```

Step 1: Skill created

create-spa-page.skill.md written to .github/skills/ and invokable as /create-spa-page

Step 2: 5 files generated

Component, CSS module, test, route, nav link — one invocation, zero convention decisions

Convention-correct output

PascalCase, CSS Modules, named exports, reference pattern — team standards baked in

Four Assets. One Directory. Infinite Reuse.



AGENTS.md

Workspace context file — gives Copilot permanent project knowledge from /init



evaluate-issue

Completeness checker — flags ticket gaps before a developer picks up the work



create-spa-page

Domain-encoded page creator — 5 files, zero convention decisions, one invocation



review-plan

Risk triage prompt — returns a severity table with a go/no-go recommendation

PART 3

Plan, Risk, and Ship

A plan is a hypothesis. /review-plan stress-tests it before code is written.

🗺️

Plan First

/plan generates tasks, files, scope, and open questions

🔍

Triage the Risk

/review-plan returns High/Med/Low with a go/no-go verdict

✅

Ship with Confidence

Risks resolved before implementation — no mid-sprint surprises

```
/plan → /review-plan → update plan → implement
↳ No-go before a line is written.
```

Code-First vs Plan-First

One path finds the architectural gap in code review. The other finds it in the plan.

❌ Code-First

1. **Pick up the ticket**
   Jump straight to implementation
2. **Write implementation**
   Discover scope as you code
3. **Open a PR**
   Reviewer finds architectural gap
4. **Rework and re-PR**
   2 extra days from late discovery

🔄 **Architectural rework discovered in code review**

✅ Plan-First with /plan

1. **Pick up the ticket**
   Generate a /plan first
2. **Review the plan**
   Scope, tasks, open questions — all explicit
3. **Run /review-plan**
   High-severity risk surfaced before code
4. **Update plan, implement**
   Risk resolved; original estimate holds

✓ **Shipped on the original estimate**

Risk found in code review → risk caught in the plan

/plan Generates a Structured Breakdown Before Any Code

/plan: Alphabet Puzzle Navigation

```
## Goal
Add nav access to a 10x10 nonogram puzzle
for each letter A-Z. Deterministic seed
= letter char code.

## Tasks
1. Generator: generateLetterPuzzle(letter)
   charCode seed + 26 unit tests
2. Pages: AlphabetIndex + extend Puzzle
   for /puzzle/alpha/:letter
3. Routes: /alphabet + /puzzle/alpha/:letter
4. Nav: add 'Alphabet' link in Nav.tsx

## Open questions
- Invalid params (/puzzle/alpha/1):
  error page or redirect?
- Persist completed puzzles to localStorage?
```

🎯 Scope made explicit

~8 files, ~200 lines estimated before a line is written

❓ Open questions surfaced

Invalid params and localStorage — decisions that cause rework if left open

👥 Reviewable before code

Any developer can read the plan and flag issues before implementation begins

The /plan Command: Natural Language, Structured Output

Copilot Chat

```
/plan let's do some big work. let's add a navigation
  item that automatically generates a 10x10 puzzle for each
  letter of the alphabet. ask questions if you aren't
  clear on what to do.
```

Conversational scoping

Describe the goal in plain language — /plan handles the task and file breakdown

Ambiguity invited

ask questions if you aren't clear surfaces open decisions before work begins

Plan, not code

The output is a reviewable plan file — implementation only starts after it's approved

/review-plan Returns a Severity Table — Not Just a List

Four risks, sorted by impact. Two are blocking. The verdict: No-go on current plan.

Determinism	Seed underspecified — identical grids not guaranteed across sessions	High
Invalid param	Error or redirect for /puzzle/alpha/1? Cascades if not decided early	Med
localStorage	Cross-cutting concern introduced if localStorage scope is decided late	Med
Test matrix	Sounds cheap but could slow the suite if the generator is complex	Low

Recommendation: No-go — resolve the determinism algorithm before Tasks 2–5 depend on it.

The /create-prompt Command: Encode the Risk Review

Copilot Chat

```
/create-prompt “review plan”. a prompt that reviews a plan
  file for technical and delivery risks. run triage and
  return hig/med/low risks to both implementation and
  delivery. report the findings in a markdown table
  so that it's easy to read.
```

⚖️ Two risk dimensions

Implementation AND delivery risks evaluated separately — not one combined list

📄 Table output by design

Markdown table format makes High/Med/Low findings scannable in any PR or doc

⚡ Invokable as /review-plan

Saved to .github/prompts/ automatically — available as a slash command immediately

The Plan Gets Updated. Then Implementation Starts.

❌ No-go: High Risk Found

Determinism algorithm unspecified
Tasks 2–5 cannot be implemented correctly without it

Two open questions need decisions before routing is built

Implementing now causes cascading rework across five tasks

💡 Plan Update

Define seed: charCode XOR prime, reproducible across sessions

Add to Task 1
Algorithm spec + 26-letter snapshot test suite

Resolve params: invalid letters redirect to /alphabet

localStorage: out of scope, deferred to follow-up

✅ Go: Implement

All open questions resolved before a line of code is written

Implementation starts clean — no mid-sprint surprises

Feature ships on the original 3-day estimate

0
architectural rework items post-review

PART 4

The Compounding Effect

The real output of the workflow is the growing toolkit, not any single feature.

📈

Each Feature Adds Assets

Every feature produces a prompt, skill, or plan template

📊

Measured Outcomes

Onboarding, ticket quality, and delivery improve per iteration

🎯

Start in 15 Minutes

Clone, run /init, commit AGENTS.md — first step done

```
After 3 months: 1 AGENTS.md + 3 prompts + 4 skills
↳ Onboarding: 5 days → 2 days.
```

Four Teams. Four Features. Four Measured Outcomes.



Onboarding

First merged PR: 5 days → 2 days. AGENTS.md answers convention questions before they're asked.



Backlog triage

15 incomplete tickets flagged in one batch; reporters fill gaps in one pass. Switches: eliminated.



Complex feature

Architectural gap caught in the plan, not in review. Feature shipped on the original estimate.



Skill library

Page creation: 100% convention-correct the sprint after encoding. Review comments: zero.

From First /init to a Fully Accumulated Team Toolkit

Each feature adds assets that compound across every developer and every feature

Week 1

Context

AGENTS.md committed — Copilot is project-aware for every developer from day one



Sprint 1

Prompts

evaluate-issue and review-plan saved — checklists become executable and shareable



Sprint 2

Skills

3–4 patterns encoded — new developers inherit team conventions instantly



TARGET

Month 2+

Full Workflow

/plan + /review-plan on every feature — risk is a planning artifact, not a review finding



The toolkit compounds — each asset makes every subsequent developer faster.

From Ad-hoc Copilot to a Repeatable Feature Workflow

Before

- Generic suggestions that miss project conventions
- Vague tickets trigger mid-sprint context switches
- Domain knowledge locked in individuals' heads
- Architectural gaps surface in code review, too late to fix cheaply

After

- AGENTS.md gives Copilot full project context from the first prompt
- evaluate-issue flags ticket gaps before a developer picks up the work
- Skills encode team patterns — invokable by any developer instantly
- /plan + /review-plan surface scope and risk before implementation

5 days → 2
Onboarding to first merged PR

Eliminated
Mid-sprint context switches from incomplete tickets

0
Architectural rework items discovered post-review

✅ WHAT YOU CAN DO TODAY

Today

- Clone any repo
- Send the /init prompt to Copilot Chat
- Review and commit the generated AGENTS.md

This Week

- Save evaluate-issue.prompt.md to .github/prompts/
- Run it against 5 tickets from the current backlog
- Note the most common gap — that's the first encoding target

This Month

- Encode 3 recurring patterns as skills in .github/skills/
- Use /plan on every non-trivial feature
- Run /review-plan before each implementation starts

🔑 Key Takeaway

The toolkit compounds — each asset saved makes the next developer faster.

REFERENCES

Official Documentation

Copilot agent mode in VS Code
Agent mode, /plan, tool invocation, and autonomous task execution

Customizing GitHub Copilot in VS Code
.prompt.md and .skill.md formats, storage locations, slash command invocation

Copilot Instructions and Context
How AGENTS.md and instruction files influence project-aware responses

Prompt Engineering for GitHub Copilot
Principles for writing effective .prompt.md files that produce consistent output

Related Content

The Agentic Journey
GitHub Actions / PR automation: the async complement to the VS Code interactive workflow

Real-world AGENTS.md example
Reference implementation from a production SPA codebase



From Repo to Feature

BUILDING TEAM TOOLS AS YOU GO WITH COPILOT IN VS CODE

30 sec

/init generates AGENTS.md — permanent context for every Copilot interaction in the project

No-go

/review-plan returns its verdict before a line of implementation code is written

5 → 2 days

Onboarding to first merged PR when Copilot has full project context from day one

What's the first pattern your team would encode as a skill?